DEEP REINFORCEMENT LEARNING · COMPANION READER

MDPs & Dynamic Programming

Adding state to the bandit: the Markov decision process, value functions, the Bellman equations, and how to solve them exactly when the model is known.

Session 3–5 (Lectures 3–5) · Read alongside the lecture slides · Sutton & Barto, Ch. 3–4

How to use this companion. Colour boxes guide you: **blue** intuition, **amber** everyday picture, **green** worked example, **red** watch out, **purple** key takeaway. Every slide formula and number is worked in full.

1 What this session covers

The bandit had one situation, repeated. Real problems have *states* that change with your actions. The **Markov Decision Process (MDP)** is the framework for this: states, actions, transition dynamics, rewards, and a discount. We define the MDP and the **return**, then the **value functions** V_π and Q_π that measure long-term goodness. The **Bellman equations** decompose value recursively, and the **Bellman optimality** equations characterise the best possible policy. Finally, **Dynamic Programming (DP)** — **value iteration** and **policy iteration** — solves the MDP *exactly* when the model is known, unified by **Generalised Policy Iteration (GPI)**.

2 The agent–environment interface and the MDP

The agent (learner and decision-maker) interacts with the environment (everything outside it) in discrete time steps: it performs an action, and the environment responds with a new state and a numerical reward. The objective is to maximise the **return** — cumulative reward over time.

An **MDP** is defined by:

- a set of **states** S and a set of **actions** A;
- **state-transition probabilities** (the **model dynamics**): $p(s' \mid s, a)$, the probability of arriving in s' after taking a in s;
- a **reward function**: the utility for the transition, often written $r(s, a, s')$ or simply $r(s, a)$;
- a start state, and possibly a terminal state.

The full dynamics combine both into one distribution, $p(s', r \mid s, a)$.

☞ The intuition

"Markov" means the future depends only on the *present* state, not the whole history. The current state captures everything you need to decide. If you know where you are now,

knowing how you got there adds nothing.

★ Everyday picture

A board game like Snakes and Ladders is Markov: your next move depends only on your current square and the dice, not on the path you took to get there. The square is a sufficient summary of the past.

2.1 Grid World: a running example

The agent lives in a grid; walls block movement. Movement is **noisy**: the intended action succeeds only 80% of the time. Taking action "North" moves North 80% of the time, but 10% West and 10% East; if a wall blocks the move, the agent stays put. Each step gives a small negative reward (battery drain); big rewards (good or bad) come at special cells. The goal: maximise accumulated reward.

✗ Watch out

Noisy transitions are why we need *expected* values. Because "North" does not reliably go north, you cannot just plan a fixed path — you must reason about the *probability* of each outcome. This is the whole reason the math has expectations and sums over s'.

2.2 The reward hypothesis

The **reward hypothesis**: all of what we mean by goals can be cast as maximising the expected cumulative sum of a scalar reward. Crucially, rewards should say *what* you want achieved, not *how* to achieve it.

✗ Watch out

Reward design is subtle and dangerous. Reward a chess agent for *capturing pieces* and it may walk into traps to grab a pawn. Reward a vacuum for *units of dirt collected* and it may dump dirt out to suck it up again. Reward the goal, never the method.

✓ Key takeaway

An MDP is $(S, A, p(s', r \mid s, a), \gamma)$ with the Markov property: the present state summarises the past. Rewards should encode the goal itself, not a recipe for reaching it.

3 Returns and discounting

The agent maximises the expected **return** G_t — a function of the future reward sequence. For **episodic** tasks (which end at a final step T, like a game or a maze run), the return is the simple sum $G_t = R_{t+1} + R_{t+2} + \cdots + R_T$. But for **continuing** tasks ($T = \infty$), that sum can be infinite. The fix is the **discounted return**:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{1}$$

where the **discount rate** $0 \le \gamma \le 1$ sets the present value of future rewards. It also yields a tidy recursion:

$$G_t = R_{t+1} + \gamma\, G_{t+1}. \tag{2}$$

☞ The intuition

γ is how far-sighted the agent is. With $\gamma = 0$ it is myopic — only the *next* reward matters. With γ near 1 it is patient — distant rewards count almost fully. Discounting also keeps the infinite sum finite and helps algorithms converge. Sooner rewards are usually worth more than later ones.

★ Everyday picture

\$100 today is worth more than \$100 next year — you could invest it, and the future is uncertain. γ is exactly this "time value of reward." A $\gamma = 0.9$ agent values a reward one step away at 90%, two steps away at 81%, and so on.

✎ Worked example: discounting a reward sequence

Take rewards $[1, 2, 3]$ over the next three steps and discount $\gamma = 0.5$.

Step 1. Apply the weights. $G = 1 + 0.5 \cdot 2 + 0.5^2 \cdot 3 = 1 + 1 + 0.75 = \mathbf{2.75}$.

Step 2. Compare with reversed rewards $[3, 2, 1]$**.** $G = 3{+}0.5{\cdot}2{+}0.25{\cdot}1 = 3{+}1{+}0.25 = \mathbf{4.25}$.

Step 3. So what. $G([3, 2, 1]) > G([1, 2, 3])$: getting the big reward *sooner* is worth more. Discounting makes the agent prefer early reward.

✎ Worked example: constant reward, infinite horizon

If the agent receives $+1$ every step forever, with $\gamma < 1$, the return is a geometric series.

Step 1. $G = 1 + \gamma + \gamma^2 + \cdots = \dfrac{1}{1-\gamma}$.

Step 2. For $\gamma = 0.9$: $G = \dfrac{1}{1-0.9} = \mathbf{10}$ — finite, even though the rewards never stop.

Step 3. So what. Discounting tames the infinite sum: a forever-stream of $+1$ is worth just 10 when $\gamma = 0.9$.

✗ Watch out

Adding a constant c to *every* reward does not change which policy is optimal in a continuing discounted task — it just shifts all returns by $c/(1-\gamma)$. But in *episodic* tasks of varying length, adding a constant *can* change the optimal policy, by rewarding longer or shorter episodes. Be careful what you add.

✓ Key takeaway

The discounted return $G_t = \sum_k \gamma^k R_{t+k+1} = R_{t+1} + \gamma G_{t+1}$ keeps infinite sums finite and makes the agent prefer sooner rewards; γ tunes how far ahead it looks.

4 Policy and value functions

A **policy** $\pi(a \mid s)$ maps states to probabilities over actions; learning means improving it. Given a policy, two **value functions** measure long-term goodness:

$$v_\pi(s) = \mathbb{E}_\pi[\, G_t \mid S_t = s \,] \qquad \text{(state-value: how good is state } s\text{?)} \tag{3}$$

$$q_\pi(s, a) = \mathbb{E}_\pi[\, G_t \mid S_t = s, A_t = a \,] \qquad \text{(action-value: how good is action } a \text{ in } s\text{?)} \tag{4}$$

They are linked. The state value is the policy-weighted average of the action values, and the action value is the immediate reward plus the discounted value of where you land:

$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s,a), \tag{5}$$

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\big[\, r + \gamma\, v_\pi(s') \,\big]. \tag{6}$$

☞ The intuition

$v_\pi(s)$ answers "if I sit here and follow π, how much reward can I expect from now on?" $q_\pi(s,a)$ answers the same but commits to one action first. Knowing q_π tells you which action is best — just pick the largest.

✓ Key takeaway

A policy maps states to action probabilities; $v_\pi(s)$ and $q_\pi(s,a)$ measure the expected return from a state (or state-action), and each can be written in terms of the other.

5 The Bellman expectation equation

Substituting one relationship into the other gives the **Bellman expectation equation** — value decomposed into immediate reward plus discounted next value:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[\, r + \gamma\, v_\pi(s') \,\big]. \tag{7}$$

In words: the value of a state equals the (discounted) value of the expected next state, plus the reward expected along the way.

☞ The intuition

This is a **consistency condition**: the value v_π must satisfy it at *every* state simultaneously. It turns a hard global optimisation — summing infinitely many future trajectories — into a recursive *local* computation: each state's value defined by its neighbours' values. With $|S|$ states it is $|S|$ linear equations in $|S|$ unknowns, solvable exactly.

★ Everyday picture

Like estimating travel time. Instead of imagining every possible route to the airport, you say: "time from here = time to the next junction + expected time from *that* junction." Each junction's estimate leans on its neighbours'. Solve them all together and every estimate is consistent.

✎ Worked example: one Bellman backup in Grid World

Take a state with the equiprobable random policy ($\pi = 0.25$ each of 4 actions), $\gamma = 0.9$, where every action moves deterministically to a neighbour with reward 0, and the four neighbours currently have values $2.3, 0.7, -0.4, 0.4$.

Step 1. Each action's target. $r + \gamma v(s') = 0 + 0.9\, v(s')$, giving $0.9 \times [2.3,\ 0.7,\ -0.4,\ 0.4] = [2.07,\ 0.63,\ -0.36,\ 0.36]$.

Step 2. Average over the policy. $v(s) = 0.25\,(2.07 + 0.63 - 0.36 + 0.36)$.

Step 3. Compute. Sum $= 2.70$; $v(s) = 0.25 \times 2.70 = \mathbf{0.675}$.

Step 4. So what. The state's value is the policy-weighted, discounted average of its neighbours — exactly the Bellman equation, one state at a time.

✓ Key takeaway

The Bellman expectation equation $v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma v_\pi(s')]$ is a consistency condition that turns a global problem into local, recursive backups — the foundation of all DP and TD methods.

6 Optimal policies and the Bellman optimality equation

A policy π is **better than or equal to** π' if $v_\pi(s) \geq v_{\pi'}(s)$ for all states s. There is always at least one **optimal policy** π_* that is best in every state (there may be several). They share the **optimal value functions** v_* and q_*. These satisfy the **Bellman optimality equations**, which replace the policy-average with a *max*:

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[\, r + \gamma\, v_*(s') \,\big], \tag{8}$$

$$q_*(s, a) = \sum_{s',r} p(s', r \mid s, a)\Big[\, r + \gamma \max_{a'} q_*(s', a') \,\Big]. \tag{9}$$

☞ The intuition

The optimal value of a state equals the expected return for the *best* action from it — not the average over a policy, but the maximum. Once you have v_* (or q_*), the optimal policy is easy: in each state, take the action that achieves the max. The hard part is computing v_*; acting on it is trivial.

✓ Key takeaway

An optimal policy π_* is best in every state; its values v_*, q_* obey the Bellman optimality equations, which take a *max* over actions. Given v_* or q_*, acting optimally is a simple greedy step.

7 Dynamic Programming

Dynamic Programming (DP) is a family of algorithms that use a *full model* of the MDP — the complete, accurate $p(s', r \mid s, a)$ — to compute optimal policies. The key idea: use value functions to structure the search. DP turns the Bellman *equations* into iterative *updates*.

✗ Watch out

DP needs the complete model. If you do not know $p(s', r \mid s, a)$, you cannot apply DP directly — you must *learn* from experience (Monte Carlo, TD), the subject of the next session. DP is the "known-model" baseline that the model-free methods imitate.

7.1 Value iteration

Value iteration repeatedly applies the Bellman *optimality* update to every state until the values stop changing (the largest change falls below a small threshold θ):

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[\, r + \gamma\, V_k(s') \,\big]. \tag{10}$$

After convergence, extract the policy greedily.

✎ Worked example: value iteration on the race-car MDP (slide numbers)

Three states (Cool, Warm, Overheated), actions Go Slow / Go Fast, $\gamma = 1$, starting from $V_0 = (0, 0, 0)$. The slides report the first sweeps for the Cool state.

Step 1. Iteration 1. Using $V_0 = 0$ everywhere, the one-step rewards give $V_1 = (2, 1, 0)$ — Cool→ 2, Warm→ 1, Overheated→ 0.

Step 2. Iteration 2. Backing up again with V_1 gives $V_2 = (3.35,\ 2.35,\ 0)$.

Step 3. Convergence. The values keep rising and settle near $V \approx (15.41,\ 14.41,\ 0)$, where the change $\Delta \approx 0.009 < \theta = 0.01$. With synchronous updates this takes about **208** sweeps without the threshold.

Step 4. Extract the policy. Pick the max-value action per state: Cool → **Fast**, Warm → **Slow**, Overheated → (terminal).

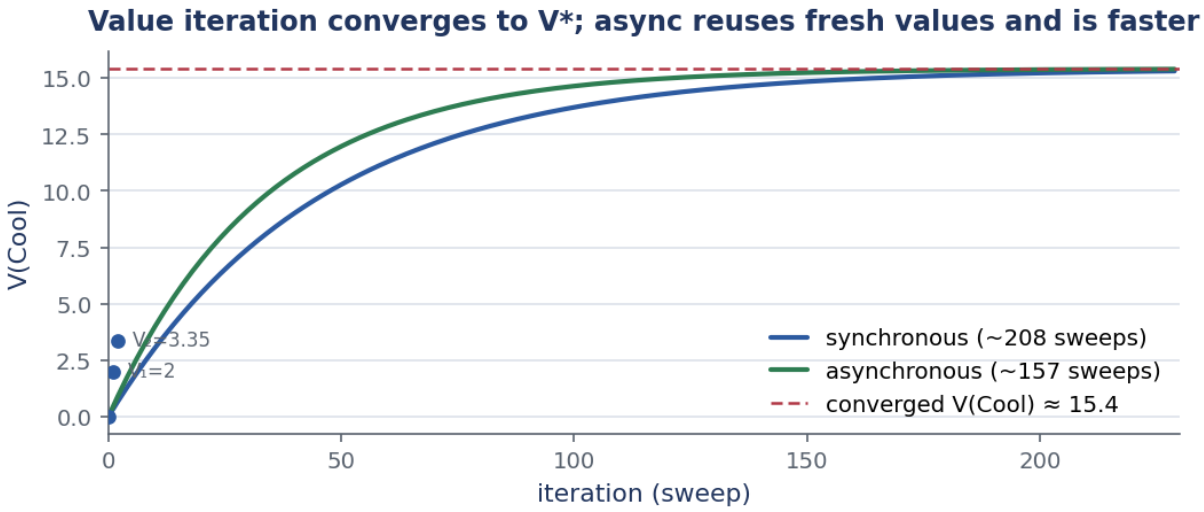

Figure 1: Value iteration on the race-car MDP. The Cool state's value climbs from 0 to 2 to 3.35 and on toward $v_*(\text{Cool}) \approx 15.4$. Asynchronous updates (green) reuse fresh values within a sweep and converge faster (∼157) than synchronous ones (∼208).

7.2 Asynchronous value iteration

Standard (synchronous) value iteration updates every state using the *previous* sweep's values. **Asynchronous** value iteration updates states in place, using the *most recent* values even within the same sweep. Any update order converges, as long as every state is visited infinitely often.

✎ Worked example: async is faster (slide numbers)

On the same race-car MDP, asynchronous updates reuse $V(\text{Cool}) = 2$ immediately when updating $V(\text{Warm})$ in the *same* sweep.

Step 1. Iteration 2, async. $V_2 = (3.75,\ 3.54,\ 0)$ — higher than the synchronous $(3.35, 2.35, 0)$, because Warm used the fresh Cool value.

Step 2. Convergence. Settles near $(15.44,\ 14.44,\ 0)$ in about **157** sweeps — versus 208 for synchronous. The fresher information speeds things up.

7.3 Policy iteration

Policy iteration alternates two steps until the policy stops changing: **policy evaluation** (compute v_π for the current policy by repeated Bellman *expectation* backups) and **policy improvement** (make the policy greedy with respect to v_π). Each new policy is provably at least as good as the last.

☞ The intuition

Value iteration is one Bellman *optimality* sweep (a max) per step, never naming the policy. Policy iteration fully evaluates a fixed policy (an average, no max — one action per state, so each pass is cheap), then improves it. Both reach v_*; they just split the work differently.

✎ Worked example: policy iteration on the race car (slide numbers)

Start with policy $\pi_0 =$ (Cool: Slow, Warm: Slow).

Step 1. Evaluate π_0. Repeated expectation backups under the fixed policy converge (asynchronously, no threshold) near $V_{225} \approx (10, 10, 0)$.

Step 2. Improve. Choosing the max-value action gives $\pi_1 =$ (Cool: **Fast**, Warm: Slow).

Step 3. Repeat. One more evaluate-improve cycle converges with $V(\text{Cool}) \approx 15.5$, $V(\text{Warm}) \approx 14.5$ — and the policy stops changing.

Step 4. So what. Policy iteration reaches the optimum in just **3** policy steps, where value iteration needs about 23 sweeps. Fewer outer iterations, but each does a full evaluation.

7.4 Value iteration vs policy iteration

Aspect	Policy Iteration (PI)	Value Iteration (VI)
Evaluation depth	Full convergence ($\Delta < \theta$)	Single Bellman sweep
Bellman update	$\sum_a \pi(a \mid s)\,[\dots]$ (expectation)	$\max_a[\dots]$
Policy update	After full evaluation	Implicitly each sweep
Policy visible?	Yes, explicit each step	Only at the end
Outer iterations	Fewer	More
Cost per iteration	More (full eval)	Less (one sweep)
Best for	Small state spaces	Large state spaces
Race car	Optimal in 3 PI steps	Optimal in 23 VI sweeps

✓ Key takeaway

DP solves a known MDP exactly: value iteration does one optimality (max) sweep per step; policy iteration fully evaluates then greedily improves a policy. PI takes fewer, heavier iterations; VI takes more, lighter ones.

8 Generalised Policy Iteration (GPI)

Generalised Policy Iteration (GPI) is the unifying schema: let *evaluation* (make V consistent with the current π) and *improvement* (make π greedy w.r.t. V) interact, at any granularity. Almost all RL methods are GPI.

☞ The intuition

The two processes *compete*: improvement changes the policy, which makes the old values wrong; evaluation fixes the values, which makes the policy no longer greedy. Yet together they drive toward the same fixed point — the optimal (π_*, v_*), where the policy is greedy w.r.t. its own values and the values are correct for the policy. Neither process must finish before the other starts.

The spectrum of GPI: policy iteration (full evaluation, then improve), value iteration (one sweep, then improve), asynchronous DP (update any states freely), and — once we drop the model — TD and Q-learning (model-free GPI) and actor–critic (parallel evaluation and improvement). The difference is only *how* evaluation is done.

✓ Key takeaway

GPI is the general pattern of interleaving policy evaluation and improvement until they reach the shared fixed point (π_*, v_*). Value iteration, policy iteration, TD, Q-learning, and actor–critic are all instances — they differ only in how evaluation is performed.

9 Session recap and the bridge ahead

We added *state* to the bandit and arrived at the MDP: $(S, A, p(s', r \mid s, a), \gamma)$ with the Markov property. The discounted return G_t made infinite-horizon goals well-defined. Value functions v_π, q_π measured long-term goodness, and the Bellman equations decomposed them recursively — expectation for a fixed policy, optimality (with a max) for the best one. Dynamic Programming then solved the MDP *exactly* when the model is known: value iteration, policy iteration, and their asynchronous variants, all unified as GPI. The catch: DP needs the full model $p(s', r \mid s, a)$. The next session asks the key question — *what if we don't know the model?* — and answers it with Monte Carlo (learn from complete episode returns), TD (learn from one-step bootstraps), and Q-learning (learn q_* directly, off-policy). GPI still applies; only the evaluation changes from known dynamics to sampled experience.